Homework 1
Computation Intelligence and its Applications in Mechatronics
Amirkabir University of Technology

Amirkabir University of Technology
(Tehran Polytechnic)

Submission deadline: **April 25, 12:00 AM**

1 RBFNN

You need to modify the following layer to use **unsupervised initialization** for `mu` and `sigma` using the KMeans algorithm on the training inputs.

1.1 Provided Layer

You are provided with the following custom `RBFLayer` implemented in TensorFlow:

```
import tensorflow as tf
from tensorflow.keras.layers import Layer
from tensorflow.keras import ops

class RBFLayer(Layer):
    def __init__(self, units):
        super().__init__()
        self.units = units

    def build(self, input_shape):
        self.mu = self.add_weight(shape=(input_shape[-1], self.units),
                                  initializer=tf.keras.initializers.RandomUniform(minval=0, maxval=5),
                                  trainable=True)

        self.sigma = self.add_weight(shape=(input_shape[-1], self.units),
                                     initializer=tf.keras.initializers.Constant(value=1.0),
                                     trainable=True)

        self.kernels = self.add_weight(shape=(input_shape[-1], self.units),
                                       initializer=tf.keras.initializers.Constant(value=1.0),
                                       trainable=False)

    def call(self, inputs):
        return self.kernels * ops.exp(-ops.square(inputs - self.mu) / (2 * ops.square(self.sigma)))
```

1.2 The Task

1. Generate a 2D dataset where the target values are defined by the following function:

$$y(x_1, x_2) = 3 \cdot e^{-\frac{(x_1-2)^2+(x_2-2)^2}{1.5}} + 2.5 \cdot e^{-\frac{(x_1+2)^2+(x_2+2)^2}{1.2}} + \sin(2x_1) \cdot \cos(x_2) + \varepsilon$$

   Where:

   - $x_1, x_2 \in [-5, 5]$
   - $\varepsilon \sim \mathcal{N}(0, 0.2^2)$ is zero-mean Gaussian noise

2. Implement the provided custom RBF layer using TensorFlow/Keras.
3. Use the **K-Means algorithm** to initialize `mu` and `sigma` in an unsupervised manner before training.
4. Train the RBFNN.
5. Try different numbers of RBF units.
6. Report and compare the loss and accuracy for each configuration.
7. Visualize the model predictions and errors for each case.
8. Analyze:
   - How does increasing the number of RBF units affect prediction accuracy?
   - Which initialization (random vs. unsupervised) performs better in terms of RMSE, convergence speed, and stability?

2 CNN

In this assignment, your task is to design, implement, and train a Convolutional Neural Network (CNN) for image classification on the CIFAR-10 dataset using TensorFlow/Keras. The goal is to build a model that achieves the highest possible test accuracy using any tools and methods you've learned so far.

2.1 Dataset: CIFAR-10

The CIFAR-10 dataset consists of 60,000 color images in 10 classes, with the following distribution:

- 50,000 training images
- 10,000 test images
- Image size: $32 \times 32 \times 3$

Each image belongs to one of the following classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Load it via:

```
from tensorflow.keras.datasets import cifar10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
```

2.2 Your Tasks

1. Preprocess the data.
2. Build a CNN model for image classification. You are free to:
   - Use **Sequential or Functional API**.
   - Apply **Dropout**, **Batch Normalization**, or other regularization techniques.
   - Use different **activation functions** or **initialization strategies**.
   - Experiment with **filter sizes**, **stride**, **padding**, and **pooling methods**.
   - Try more advanced ideas such as **residual connections**, **inception-like blocks**, or custom layers.
3. Train your model on the training set and evaluate it on the test set.
4. Use appropriate callbacks (e.g., `EarlyStopping`, `ModelCheckpoint`, or `ReduceLROnPlateau`) to avoid overfitting and improve convergence.
5. Provide a performance summary:
   - Final **Test Accuracy**
   - Training/Validation Accuracy and Loss plots
   - Number of parameters in your model
   - Confusion Matrix (optional, for detailed error analysis)
6. BONUS: Use `TensorBoard` to monitor the training process and visualize your architecture and metrics.

Extra Point

The group that achieves the highest test accuracy for the CNN task among all submissions will receive an extra point for this assignment.

Submission Guidelines

- Homeworks must be completed in **groups of four**.
- Each group must submit a single file containing a **PDF report**, including answers to discussion questions, visualizations, and code explanations, and a **Jupyter Notebook**, containing all code and results.